Statistik-Labor Testat Aufgabe 1

Hinweise:

- Bitte überprüfen Sie Ihre Resultate vor der Abgabe Ihre Ergebnisse mit den Teilergebnissen aus der Datei ***Teilergebnisse_xy.txt***. In der Datei bekommen Sie jeweils die ersten beiden Nachkommastellen des Durchschnittsalters.
- Die Unterlagen sind im **pdf-Format** in Moodle hochgeladen abzugeben. Richtige Lösungen werden nicht mehr an Sie zurückgegeben. Eine Abgabe per Email ist nicht möglich.
- Bitte geben Sie Ihrer Abgabedatei einen **sinnvollen Dateinamen**, z.B.: Testat1_*Nachname_Matrikelnummer*_v*X*. *(Wobei X für die Abgabeversion 1 bis 3 steht.)*

Aufgabenstellung:

Sie sollen für die Personalabteilung der NANUAG die Altersverteilung der Beschäftigten analysieren. In der Datei **sr_aufg_1_xy.txt** im Unterverzeichnis **Endziffer_xy** (*„xy" steht für die beiden Endziffern Ihrer Matrikelnummer*) finden Sie die dafür erforderlichen Daten. Diese Datei enthält folgende fünf Felder, die durch Semikolons getrennt sind: Personalnummer, Anrede („Herr" oder „Frau"), Vorname, Nachname, Geburtsdatum.

a) Daten einlesen

Lesen Sie die Daten in **Python** bzw. ein **Jupyter Notebook** ein. Speichern Sie sie in einem **Pandas DataFrame** ab.

Tipp: Bitte beachten Sie bei der Abgabe, dass Sie auf keinen Fall die Liste mit den 100 Datensätzen ausgeben! Verwenden Sie dafür zum Beispiel die `dataframe.head()`-Funktion.

```
import pandas as pd

#ToDo: 1. Ersetzen Sie im Dateinamen xy durch entsprechende
Matrikelnummer
#      2. Lassen Sie sich alle Daten anzeigen, um "fehlerhafte" Daten
zu erkennen; vor Abgabe bitte nur z.B. die ersten 5

data = pd.read_csv('sr_aufg_1_19.txt', sep=';',
            encoding='unicode_escape', na_values='.', header = None,
            names=['Pers.Nr.', 'Anrede', 'Vorname', 'Nachname',
'Geburtsdatum'])



data.head()

   Pers.Nr. Anrede Vorname Nachname Geburtsdatum
0         1   Frau  Marion    Meier   25.09.1973
```

```
1          2   Herr  Ulrich  Meister   03.09.1964
2          3   Frau   Doris   Schwab   23.09.1965
3          4   Frau   Beate  Schäfer   10.10.1970
4          5   Herr     Tim   Schwan   19.01.1974
```

b) Daten bereinigen

Führen Sie eine Datenbereinigung durch. Listen Sie in einer Tabelle auf, welche (Daten-)Sätze Sie gelöscht und welche Sätze Sie wie korrigiert haben.

Tipp: Sie können hierzu die Python Funktionen `dataframe.drop()` und `dataframe.at()` verwenden. ***Beispiellösung:*** Tabelle mit den gelöschten & korrigierten Daten

```python
Datenb = pd.DataFrame()    # DataFrame für die Tabelle mit den Fehlern
indizes = [24, 28, 37, 38, 49, 55, 71, 77, 85, 89, 95, 103]

for i in indizes:
    if i < len(data):
        Datenb = pd.concat([Datenb, data.loc[[i]]], axis=0)

Datenb['Datensatz?'] = None
Datenb['Typ des Fehlers'] = None

# Diese Datensätze werden gelöscht
loeschungen = {
    24: "kein Datum angegeben",
    28: "Zeitreisende? Geb.Jahr 3789",
    37: "Jahr fehlt",
    38: "Jahr und Monat fehlen",
    49: "Jahr und Monat fehlen",
    55: "Zeitreisende? Geb.Jahr 25400",
    71: "Zeitreisende? Geb.Jahr 3241",
    85: "Zeitreisende? Geb.Jahr 2358",
    95: "kein Datum angegeben",
    103: "Unsinn"
}

for index, fehler in loeschungen.items():
    if index in data.index:
        Datenb.at[index, 'Datensatz?'] = "Löschen"
        Datenb.at[index, 'Typ des Fehlers'] = fehler
        data = data.drop(index)

# Diese Datensätze werden korrigiert
korrekturen = {
    77: ("Format des Datums", "22.02.1973"),
    89: ("Format des Datums", "20.02.1969")
}
```

```python
for index, (fehler, neues_datum) in korrekturen.items():
    Datenb.at[index, "Datensatz?"] = "Korrektur"
    Datenb.at[index, "Typ des Fehlers"] = fehler
    data.at[index, 'Geburtsdatum'] = neues_datum

Datenb
```

```
     Pers.Nr. Anrede      Vorname   Nachname         Geburtsdatum
Datensatz?  \
24         25   Herr       Moritz        Alt
Löschen
28         29   Frau        Marie     Schwan           28.01.3789
Löschen
37         38   Herr        Bernd    Schnell               09.12.
Löschen
38         39   Frau        Tanja    Winkler                  26.
Löschen
49         50   Herr      Günther    Büchner                  26.
Löschen
55         56   Herr        Frank    Winkler          31.10.25400
Löschen
71         72   Herr       Rudolf      Kraus           21.03.3241
Löschen
77         78   Herr       Stefan       Rapp  February 22nd, 1973
Korrektur
85         86   Herr        Josef  Schneider           14.06.2358
Löschen
89         90   Herr  Hans-Jürgen   Hartmann  am 20. Februar 1969
Korrektur
95         96   Frau      Melanie      Frank                 xxxx
Löschen
103       104   Herr      Torsten   Hoffmann           UfkIBMhAZY
Löschen

                   Typ des Fehlers
24           kein Datum angegeben
28    Zeitreisende? Geb.Jahr 3789
37                     Jahr fehlt
38          Jahr und Monat fehlen
49          Jahr und Monat fehlen
55   Zeitreisende? Geb.Jahr 25400
71    Zeitreisende? Geb.Jahr 3241
77               Format des Datums
85    Zeitreisende? Geb.Jahr 2358
89               Format des Datums
95           kein Datum angegeben
103                        Unsinn
```

c) Alter bestimmen

Berechnen Sie das Alter der Beschäftigten zum Stichtag 31.12.2005. Mit „Alter“ ist hier die Anzahl der vollendeten Lebensjahre gemeint. *Beispiel:* Wer am 30.06.1975 geborgen ist, ist zum Stichtag 30 Jahre alt. (Niemand würde von sich sagen, er wäre „30,5 Jahre alt“.) Folglich wird diese Person in der Alterklasse 21 – 30 Jahre mitgezählt.

Tipp: In Python ist hierzu die Pandas-Bibliothek mit ihre Funktion `pandas.to_datetime()`, sowie `pandas.year` nützlich.

```
from datetime import datetime

stichtag = datetime(2005, 12, 30)  # Fixes Datum

def berechne_alter_stichtag(geburtsdatum_str):
    try:
        geburtsdatum = datetime.strptime(geburtsdatum_str, "%d.%m.%Y")
        alter = stichtag.year - geburtsdatum.year - ((stichtag.month,
stichtag.day) < (geburtsdatum.month, geburtsdatum.day))
        return alter
    except:
        return None

data['Alter'] = data['Geburtsdatum'].apply(berechne_alter_stichtag)

data.head()
```

```
   Pers.Nr. Anrede Vorname Nachname Geburtsdatum  Alter
0         1   Frau  Marion    Meier   25.09.1973     32
1         2   Herr  Ulrich  Meister   03.09.1964     41
2         3   Frau   Doris   Schwab   23.09.1965     40
3         4   Frau   Beate  Schäfer   10.10.1970     35
4         5   Herr     Tim   Schwan   19.01.1974     31
```

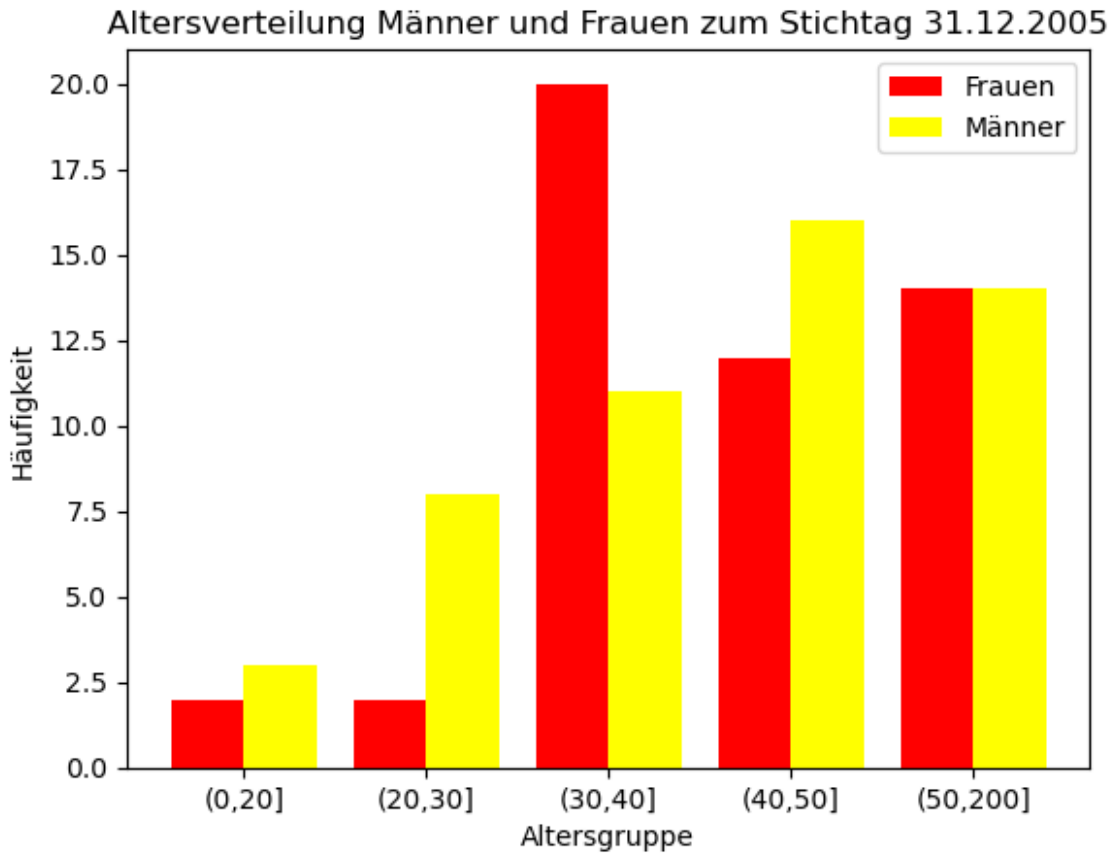
d) Säulendiagramm mit Altersverteilung

Stellen Sie die Altersverteilung der Beschäftigten in einem gruppierten Säulendiagramm dar, in dem Männer und Frauen getrennt ausgewiesen werden. Es sollen die Altersklassen bis (einschließlich) 20 Jahre, 21 – 30 Jahre, 31 – 40 Jahre, 41 – 50 Jahre und über 50 Jahre gewählt werden.

Tipp: Sie benötigen hierfür die Python Bibliothek **Matplotlib** und **NumPy** sowie aus **Pandas** die Funktion `value_counts()`. ***Beispiellösung:*** Säulendiagramm mit männlichen und weiblichen Beschäftigten

```
import numpy as np
import matplotlib.pyplot as plt
```

```
Altersgrenzen = [0, 20, 30, 40, 50, 200]

Altersklassen = ['(0,20]', '(20,30]', '(30,40]', '(40,50]',
'(50,200]']

# Neue Spalte Altersklassen wird hinzugefügt und gefüllt
data['Altersklasse'] = pd.cut(data['Alter'], Altersgrenzen, labels=
Altersklassen)

frauen_counts = data[data['Anrede'] == 'Frau']
['Altersklasse'].value_counts(sort=False)     # für jede Altersklasse
werden die Frauen/Männer gezählt
maenner_counts = data[data['Anrede'] == 'Herr']
['Altersklasse'].value_counts(sort=False)



fig , ax = plt.subplots()

x = np.arange( len( data['Altersklasse'][0:5]))

plt.bar( x - 0.2  , frauen_counts [0:5] , width=0.4, color='red',
label='Frauen')
plt.bar( x + 0.2  , maenner_counts [0:5] , width=0.4, color='yellow',
label='Männer')

plt.xticks( x , Altersklassen )          # darstellungseinstellungen
plt.xlabel('Altersgruppe')
plt.ylabel('Häufigkeit')
plt.title('Altersverteilung Männer und Frauen zum Stichtag
31.12.2005')
plt.legend()

plt.show()
```



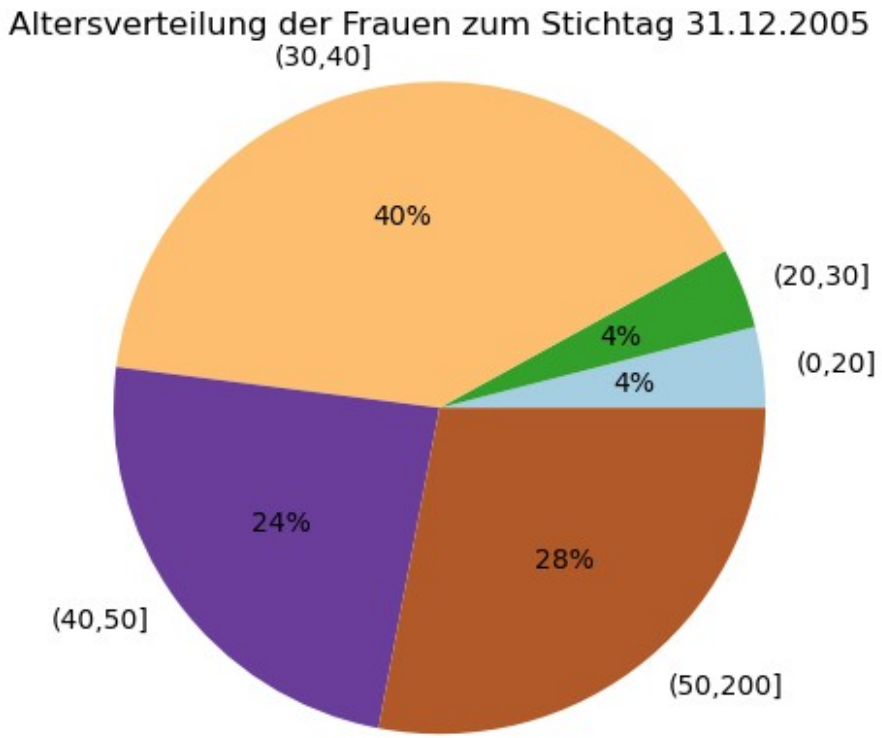
e) Kreis-/Tortendiagramm der Altersverteilung

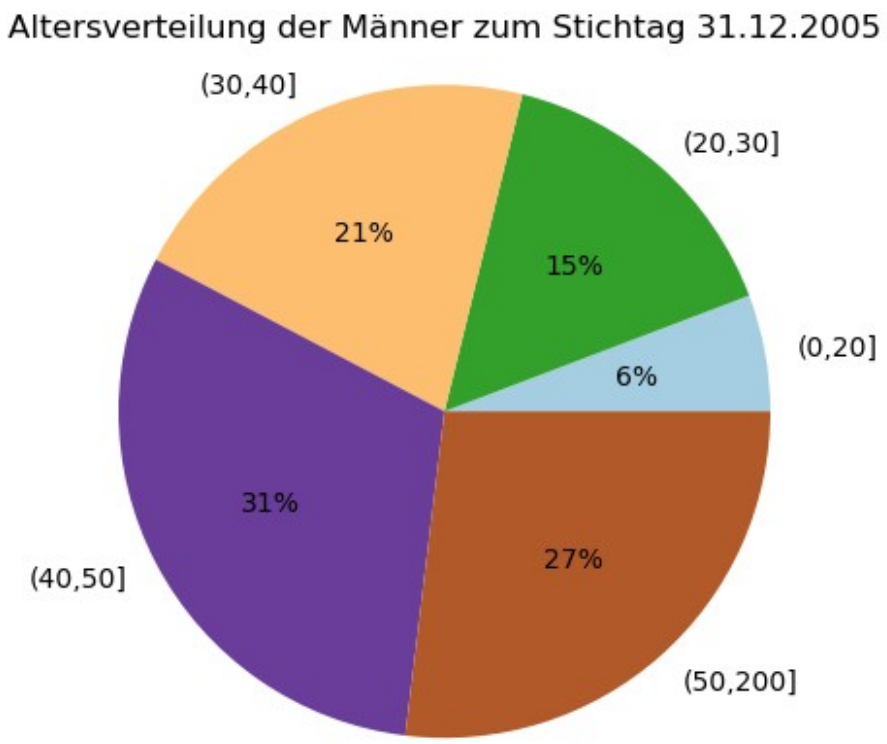
Stellen Sie die Altersverteilung der männlichen Beschäftigten und der weiblichen Beschäftigten in je einem Kreisdiagramm („Tortendiagramm") dar. Die Kreissektoren sind mit den zugehörigen Prozentzahlen zu beschriften.

Beispiellösung: Tortendiagramm der weiblichen Beschäftigten

```
# Kreisdiagramm Frauen
farben = plt.cm.Paired(np.linspace(0, 1, len(frauen_counts)))    # für
gut unterscheidbare Farben

plt.pie(frauen_counts , labels=Altersklassen , colors=farben ,
autopct='%1.0f%%' )  # für ganzzahlige Prozentwerte

plt.title('Altersverteilung der Frauen zum Stichtag 31.12.2005')
plt.axis('equal')

plt.show()


# Kreisdiagramm Männer
plt.pie(maenner_counts , labels=Altersklassen , colors=farben ,
```

```
autopct='%1.0f%%' )

plt.title('Altersverteilung der Männer zum Stichtag 31.12.2005')
plt.axis('equal')

print()   # für mehr abstand
plt.show()
```





f) Kennzahlen

Berechnen Sie **Mittelwert**, **empirische Varianz**, **empirische Standardabweichung**, **Median** und **Spannweite** der Alterszahlen:

- für alle Beschäftigen und
- getrennt für Männer und Frauen.

Tipp: Bitte auf max. 4 Nachkommastellen runden! ***Beispiellösung:*** Übersicht der Kennzahlen

```
kennzahlen = {'gesamt': [data['Alter'].mean(),
                         data['Alter'].var(),
                         data['Alter'].std(),
                         data['Alter'].median(),
                         data['Alter'].max() - data['Alter'].min()],

               'Frauen': [data[data['Anrede'] == 'Frau']
['Alter'].mean(),
                        data[data['Anrede'] == 'Frau']['Alter'].var(),

                        data[data['Anrede'] == 'Frau']['Alter'].std(),

                        data[data['Anrede'] == 'Frau']
['Alter'].median(),
                        data[data['Anrede'] == 'Frau']['Alter'].max()
- data[data['Anrede'] == 'Frau']['Alter'].min()] ,
```

```
              'Männer': [data[data['Anrede'] == 'Herr']
['Alter'].mean(),
                         data[data['Anrede'] == 'Herr']['Alter'].var(),

                         data[data['Anrede'] == 'Herr']['Alter'].std(),

                         data[data['Anrede'] == 'Herr']
['Alter'].median(),
                         data[data['Anrede'] == 'Herr']['Alter'].max()
- data[data['Anrede'] == 'Herr']['Alter'].min()]
             }

# Umwandlung dict in pd-Dataframe mit neuer Zeilen-Indexierung (statt
vorheriger Durchnummerierung)
kennzahlen_Frame = pd.DataFrame(kennzahlen, index = ['Mittelwert',
'Varianz', 'Std.-Abw.', 'Median', 'Spannweite'])
kennzahlen_Frame.round(4)
```

	gesamt	Frauen	Männer
Mittelwert	42.2451	42.1000	42.3846
Varianz	140.7215	115.8061	167.3786
Std.-Abw.	11.8626	10.7613	12.9375
Median	43.0000	42.0000	43.0000
Spannweite	46.0000	46.0000	44.0000